



Fast and Memory-Efficient Approximate Minimum Spanning Tree Generation for Large Datasets

Mahmood K. M. Almansoori¹ · Andras Meszaros^{1,2} · Miklos Telek^{1,2}

Received: 17 September 2023 / Accepted: 11 March 2024 / Published online: 21 June 2024
© The Author(s) 2024

Abstract

Conventional minimum spanning tree (MST) algorithms typically start by creating a distance matrix of the $n(n-1)/2$ pairs of data points, leading to a time complexity of $O(n^2)$. This initial step poses a computational bottleneck. To overcome this limitation, we present a novel method that constructs an initial random k -neighbor graph and optimizes this graph by employing a crawling technique to efficiently approximate the k Nearest Neighbors (k NN) graph. This crawling approach allows us to approximate the closest neighbors of each node. Subsequently, the approximate k NN graph is utilized to build an initial approximate MST and iteratively refine it by the same crawling process. Using this approach, an approximate MST can be obtained for a data set of size n with empirical cost around $O(n^{1.07})$ and a minimal $O(n)$ memory consumption. In contrast to spatial tree-based approaches, the presented method also scales well to high dimensional data. We have shown that the proposed approach achieves such a level of performance with only a marginal accuracy reduction between 0.5% and 6%. These qualities make it an excellent candidate for approximate MST calculation for high-dimensional, large data sets.

Keywords Large datasets · High-dimensional data · Approximate MST · Memory-efficient

1 Introduction

Minimum spanning tree (MST) algorithms have numerous applications in the domain of big data due to their ability to extract essential structures from large data sets. Some of the notable applications are clustering, network design and optimization, and anomaly detection. The goal of clustering is to group similar data points into clusters while keeping dissimilar points separate. MST is a graph-theoretic concept that can be used to identify groups of closely related data points that can be clustered together [1–4]. MST is a tree that spans all the nodes of a graph while minimizing the total weight of the edges. In the context of clustering, the nodes

represent data points, and the weights of the edges represent the similarity or dissimilarity between the points. A low-weight edge between two nodes indicates a high similarity, while a high-weight edge indicates a low similarity.

MST can be used to identify clusters of data points by partitioning the tree into subtrees, where each subtree represents a cluster [5, 6]. The subtrees can be identified by cutting the edges of the tree at a certain threshold weight. The resulting subtrees consist of groups of closely related points that can be clustered together. Additionally, MST can also be used as a pre-processing step for clustering algorithms [7–9]. It can reduce the complexity of clustering algorithms by focusing on the most relevant pairwise relationships and filtering out noisy or less informative connections. The resulting MST can serve as a compact representation of the data, facilitating faster and more efficient clustering.

Although numerous algorithms construct MST accurately and efficiently, these algorithms typically require a graph that reflects the relationships among the data points [4, 10, 11]. However, building such a graph can be computationally expensive. In recent years, many algorithms have utilized the k Nearest Neighbors (k NN) graph to construct the MST. The k NN graph approximates the underlying data relationships by connecting each data point to its k nearest neighbors. By

✉ Mahmood K. M. Almansoori
almansoori@hit.bme.hu

Andras Meszaros
meszarosa@hit.bme.hu

Miklos Telek
telek@hit.bme.hu

¹ Department of Networked Systems and Services, Budapest University of Technology and Economics, Budapest, Hungary

² ELKH-BME Information Systems Research Group, Budapest, Hungary



leveraging the k NN graph, MST construction algorithms can effectively capture the data structure and generate an MST that represents the relationships among the data points. This approach offers a computationally feasible and scalable solution for constructing MSTs in large data sets.

There are many algorithms that use k NN for classification and regression tasks in the field of machine learning and data mining. These algorithms are nonparametric (do not make any assumptions about the underlying data distribution) and instance-based (relies on the entire data set during the prediction phase) learning algorithms [12, 13]. However, finding the exact k nearest neighbors in high-dimensional data sets is computationally expensive, especially for large data sets [14], since the brute force approach, which involves computing the distance between all point pairs in the data set, has a time complexity of $O(n^2)$. Therefore, approximate k NN search techniques have gained significant attention since they are able to find near neighbors in large data sets much more efficiently [15].

While the approximate k NN intends to approximate the true k NN closely, the two can be different due to the reduced computational complexity. The accuracy of approximate k NN search techniques for the entire data set can be measured by the difference between the approximate k NN distances and the true k NN distances:

$$\text{Accuracy} = \sum_{i=1}^n \sum_{j=1}^k \tilde{D}_{ij} - D_{ij}, \quad (1)$$

where n is the number of data points in the data set, k is the considered number of nearest neighbors, D_{ij} is the true distance to the j th nearest neighbor of the i th data point, and $\tilde{D}_{ij}$ refers to the distance of the i th data point to its j th nearest neighbor in the approximate k NN.

These techniques aim to get a balance between accuracy and computational complexity, allowing for faster search times while still providing reasonably accurate results. Several methods have been developed to address the approximate k NN search problem. The three main categories of such methods are spatial-tree-based, hashing-based, and graph-based methods.

Spatial tree-based methods partition the space into smaller regions or nodes, creating a hierarchical structure. The goal is to efficiently organize the data so that spatial queries like nearest-neighbor searches can be performed quickly. Examples of spatial trees include KD-tree [16], Ball-tree [17, 18] and K -Means tree [19]. The construction of a KD-tree involves recursively partitioning a set of points in a K -dimensional space. A splitting axis is chosen, often alternating between dimensions, and data points are separated based on their values along that axis. The median point along the chosen axis becomes the current node, and the process

continues for the left and right subtrees until all points are organized into the tree structure.

Spatial-tree-based methods offer fast searching by traversing the tree structure. The advantages of tree-based methods include logarithmic search complexity, good performance for low-dimensional data, and support for range searches. However, they can struggle with high-dimensional data due to the curse of dimensionality and may suffer from unbalanced trees or inefficiency when the data distribution is skewed [20].

Hashing-based methods [21] map high-dimensional data points into lower-dimensional binary codes, enabling efficient indexing and searching. They offer fast query processing and are effective for large-scale data sets. Using the Hamming distance, these methods allow for efficient retrieval of approximate k NN. They have many advantages, including sublinear search complexity, scalability, and the ability to handle high-dimensional data [20]. However, there is a trade-off between accuracy and speed, as hashing-based methods may not always guarantee that all nearby vectors will fall into the same or nearby buckets, which can affect the accuracy of the search [22].

In approximate k NN search, graph-based methods represent data points as nodes in a graph and establish edges based on their distances [15]. These methods utilize graph traversal techniques to find approximate k NN. The advantages of graph-based methods include handling both sparse and dense data, flexibility in defining proximity relationships, and good performance for high-dimensional data [23, 24]. They can also support incremental updates to the graph. However, constructing and maintaining the graph can be computationally expensive, and the search complexity can be higher than with tree-based methods.

This paper provides a method for the construction of the approximate k NN graph, which is customized to generate a high-quality approximate MST. The proposed method is able to derive an approximate MST that is close to the exact MST with a lower computational cost, thereby enhancing the overall performance and efficiency of clustering algorithms, such as Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) [25], for which MST generation is a crucial step. The algorithm scales well to high-dimensional and large data sets, where not only exact methods, but also some approximate methods become computationally infeasible. Two versions of our algorithm are presented. The first version stands out for its memory efficiency. The second version maintains a balance between memory efficiency and execution time. It stores a reduced number of the computed distances during the optimization process, thereby ensuring a shorter execution time than the memory-efficient version and a lower memory consumption than the brute force method.

The rest of the paper is structured as follows. In Sect. 2, the relevant solutions from the literature are presented. The

proposed method is detailed in Sect. 3. Results and discussions are introduced in Sect. 4. Finally, a conclusion of the paper is given in Sect. 5.

2 Existing MST Generation Methods

In graph theory, an MST is a subgraph of a given undirected graph that connects all the data points in a data set while minimizing the total sum of edge weights. The construction of an MST must follow the cut and cycle properties. The cut property of MSTs states that the lightest edge crossing any cut must be part of the MST, while the cycle property dictates that the heaviest edge in any cycle cannot be in the MST.

MSTs are employed in clustering and data analysis algorithms to identify patterns and relationships within large data sets. Constructing an MST on data points makes it possible to identify clusters and hierarchies, enabling efficient data exploration and visualization. MSTs can indeed serve as the primary and exclusive technique for unsupervised data clustering. For instance, MST plays a crucial role as a fundamental step within the HDBSCAN clustering framework. An MST can be generated using exact or approximate methods.

2.1 Exact Methods

Constructing an MST from a data set of n points in a d -dimensional space consists of two steps. First, a weighted undirected full graph is constructed from the data set, where each node represents a data point, and edges between nodes convey the similarity between the points. Second, the MST is constructed from this full graph. The process of constructing an MST can be achieved using various algorithms, such as Prim's algorithm [26], Boruvka's algorithm [27], or Kruskal's algorithm. Here is a general outline of the procedure for building an MST from a full graph using Kruskal's algorithms [26, 28]:

- Create a set for each node, initially containing only that node.
- Create a list of all edges in the graph.
- Sort the list of edges in the non-decreasing order of their weights.
- Select the first edge from the sorted list that has not been selected yet. On the condition of not forming a cycle, add this edge to the current MST and merge the sets of the two endpoints of the edge.
- Repeat the last step until the MST contains $(n - 1)$ edges, where n is the number of nodes in the full graph (identical to the size of the data set).

The authors in [29] introduce the Dual-Tree Boruvka algorithm, which efficiently computes the MST using a span-

ning forest approach. The algorithm employs a dual-tree strategy, utilizing either a KD-tree or a cover tree to identify nearest neighbor pairs for components within the forest simultaneously. The algorithm employs distance-based pruning mechanisms, ensuring accurate identification of nearest neighbors by eliminating points that cannot possibly form MST edges. This approach minimizes unnecessary distance calculations by leveraging space partitioning trees and properties of cover trees, minimizing unnecessary distance computations while efficiently exploring the tree structure when constructing the MST. Although the authors purport a time complexity of approximately $O(N \log N)$, the performance of the algorithm depends on space partitioning trees. Due to this feature, its efficiency is reduced in high-dimensional data. Experimental results (Sect. 4) show that its scalability needs to be improved regarding the dimensionality of the data.

The process of building the exact MST from the full graph has a time complexity of $O(n^2)$. This quadratic complexity poses challenges in the case of large data sets due to the significant time and memory requirements. Therefore, more efficient algorithms or specialized techniques are pursued to address this limitation and enable cluster analysis on very large data sets at the price of computing an approximate MST with a higher weight than the MST, which we refer to exact MST in the sequel for distinction.

2.2 Approximate Methods

Many algorithms initiate the construction of the approximate MST by generating a graph with a reduced number of edges, such as k NN graphs. When the $O(n^2)$ time complexity of exact k NN graph generation is infeasible due to the size of the data set, efficiently generated approximate k NN graphs can be used. Many of the efficient approximate k NN graph generation procedures are based on spatial trees since the cost of their construction is low. For example, the time complexity of constructing a KD-tree from a d dimensional data set is $O(d \cdot n \log n)$ and to generate an approximate k NN graph from a balanced KD-tree is $O(n \cdot k \log n)$.

The price of this reduced computational cost is the reduced accuracy. The sum of the weights of the generated approximate k NN graphs is higher than that of the exact k NN graph, but the structure of the approximate k NN graphs is usually quite similar to the exact k NN graph, thus the classification based on approximate k NN graphs is similar to the one based on an exact k NN graph.

One of the most efficient approximate MST generation methods using KD-tree is the Well Separated Pair Decomposition- (WSPD) [30–32] based method proposed in [28]. In computational geometry, a WSPD of a data set X in d -dimensional space is a sequence of pairs of subsets (A_i, B_i) . Each pair maintains a significant distance between



Table 1 Data sets of different sizes and dimensions

Data set	Type	Dimensions (d)	Size (n)
Speech	real	400	3,686
Miss America		16	6480
MNIST		100	7603
Shape		544	28,775
House		3	34,112
Shuttle		9	49,097
Audio		192	54,387
Europe		2	169,308
Celeba		39	202,599
Corel		14	662,317
Unbalanced	synthetic	2	6500
Birch1		2	10^5
Make_moons		2	40
Make_blobs		2–1000	10^3 – 10^6

subsets, and for any two distinct points $p, q \in X$, there is exactly one pair that separates them, placing p in A_i and q in B_i . WSPD helps organize points for efficient geometric algorithms by ensuring separation and clear pairwise relationships. A_i and B_i are considered ϵ -well-separated if

$$\epsilon \cdot \text{distance}(A_i, B_i) \geq \max(\varnothing(A_i), \varnothing(B_i))$$

where the distance of set A_i and B_i is defined as the minimum distance between the bounding boxes of set A_i and B_i , the $\varnothing(A_i)$ is the length of the diagonal of the bounding box of the A_i set, and ϵ is a hyperparameter.

In [28], Wang, et al. propose a fast approximate MST construction approach utilizing the WSPD technique. Their proposed method is composed of three steps:

1. building a spatial tree (they use KD-tree),
2. building a WSPD based on the spatial tree,
3. connecting pairs of WSPD using Bichromatic Closest Pair (BCCP) [30], which involves finding the pair of points (a_i, b_i) that have the smallest distance between set A and set B , where $a_i \in A$ and $b_i \in B$.

In [28], the authors present a parallel version of this approach as a step in the parallel HDBSCAN clustering process.

The graph obtained through WSPD has only $O(n \log n)$ edges and is computed in $O(dn \log n + \epsilon^d d^{d/2} n)$ time [33]. However, steps (1)–(3) are considerably slower than the method proposed in this paper because WSPD relies on spatial trees, which scale poorly with the dimensionality of the data [33]. Additionally, adapting this algorithm for an incremental extension of the data set is non-trivial.

In this work, the WSPD-based method proposed in [28] is considered to be the most efficient among the existing

approximate MST generation methods [34], and is compared with the proposed method in the numerical section.

In [35], an enhanced and efficient implementation of HDBSCAN called FISHDBC is introduced. The authors employ a technique referred to as Hierarchical Navigable Small World (HNSW) to achieve this improvement. While HNSW is efficient in approximate k NN search, the construction of HNSW is less efficient [15]. The approach also demands considerable memory resources [36]. Furthermore, the method may approximate the MST by a spanning forest initially, and the creation of a single approximate MST from this forest is improperly documented in [35].

In [37], Wang, et al. present a fast approximate MST-based clustering algorithm that employs a divide-and-conquer strategy. This method efficiently identifies long edges in the initial stages to decrease the required distance computations by utilizing the cut and cycle properties of MSTs. However, the worst-case complexity of the algorithm in [37] remains $O(n^2)$, suggesting that it could face performance challenges when dealing with large data sets.

A K -means-based fast approximate MST computation is proposed in [38]. The solution consists of the following stages:

- $\lfloor \sqrt{n} \rfloor$ partition
 - dividing the data set into $\lfloor \sqrt{n} \rfloor$ clusters (using the K -means method)
 - computing the exact MST for each cluster
 - connecting these MSTs into an approximate MST of the entire data set
- $\lfloor \sqrt{n} \rfloor - 1$ partition
 - repeating the same step with $\lfloor \sqrt{n} \rfloor - 1$ clusters

- the two approximate MSTs are combined into a single approximate MST.

The expected time complexity of the procedure is $O(n^{1.5})$. However, the actual execution time might vary highly. This variation is due to the strong dependence on the distribution of partitions generated from the K -means algorithm. The computational complexity of the step of computing the exact MST for each cluster depends on the size of the largest cluster. A strongly unbalanced cluster structure might result in significant running time.

Jothi et al. propose a recursive approach for clustering data in [19]. The data set is recursively divided into a tree of clusters by repeatedly splitting the data set into two sets until the partition size meets a cluster size limit. This process creates a tree where the leaf nodes represent the final partitions of the data set. A full subgraph is then generated for each partition. The following steps involve identifying neighbor partitions and boundary points. Finally, the subgraphs are connected, and an approximate MST is constructed from them, achieving an expected time complexity of $O(n^{1.5} \log n)$ [39]. However, the accuracy of the final approximate MST heavily depends on the quality of the inter-set edges calculations used to identify neighboring partitions. This dependency may lead to less accurate approximate MSTs, especially when dealing with complex, high-dimensional data sets [40].

3 The Proposed Method

The goal of this paper is to construct an approximate MST efficiently, which is similar to the exact MST, from a data set denoted as X , where $X = \{x_1, x_2, \dots, x_n\}$, comprises n data points in a d -dimensional space.

The proposed method is composed of the following steps:

- Random k -neighbor graph generation.
- Optimization of the random k -neighbor graph to obtain an approximate k NN graph
- Generation of an approximate MST from the approximate k NN graph
- Optimization of the approximate MST.

3.1 Random k -Neighbor Graph Generation

The initial step in constructing an approximate k NN graph is creating a random directed graph with $(n \cdot k)$ edges, such that each node has k neighbors. The weight associated with the edges between node x_i and x_j is the distance between them. This way, the weights of edge (x_i, x_j) and edge (x_j, x_i) are identical. Algorithm 1 generates a random directed graph by assigning k random data points to each node as neighbors. Each node is represented by a unique key in the graph dic-

tionary, and the corresponding value is a list of k indices representing its neighbors. A random graph of this nature is constructed with a time complexity of $O(n \cdot k)$.

3.2 Optimizing the Random k -Neighbor Graph

The optimization of the random graph generated by Algorithm 1 aims to eliminate long edges based on the procedure proposed in [41]. The elementary step of the procedure is as follows. If x_j is a neighbor of x_i and x_z is a neighbor of x_j and $distance(x_i, x_j)$ is greater than $distance(x_i, x_z)$, then x_j is replaced by x_z in the neighbors list of x_i . This elementary step is performed for all nodes and all neighbors of neighbors according to the pseudo-code in Algorithm 2. Since the replaced list of neighbors changes the structure of the graph, further iterations of the same steps might further refine the list of neighbors.

The random k -neighbor graph optimization process, according to Algorithm 2, can be continued until either convergence is achieved (where no further neighbors of neighbors are found to be closer to a given node) or a pre-determined number of iterations is reached. Two versions of the random k -neighbor graph optimization process are introduced with different iteration policies based on hyper-parameters Δ and δ ($\Delta \gg \delta$) as follows:

- *Long k -neighbor graph optimization* (Algorithm 3): In this procedure, the random k -neighbor graph is optimized according to Algorithm 2 until either converge or reaching Δ iterations. At this point, an undirected graph is generated from the directed k -neighbor graph such that one of the edges is dropped in case of a node pair connected with two oppositely directed edges. This way, the number of undirected edges is between $n \cdot k/2$ and $n \cdot k$, and the weights of the edges are the distances between the corresponding nodes. The approximate MST is obtained as the exact MST of this undirected graph. (The obtained MST is suboptimal because only a limited number of edges of the full graph is used to compute it.)
- *Short k -neighbor graph optimization* (Algorithm 4): In this procedure, the random k -neighbor graph is optimized by Algorithm 2 only δ times (if it does not converge before), and after that, an undirected graph and its MST of $n - 1$ edges is generated as in the previous case. In contrast to the *long k -neighbor graph optimization* method, the obtained approximate MST is further optimized in this procedure. First, the undirected approximate MST is transformed to a directed graph of $2n - 2$ edges such that the undirected edges are substituted with directed edges in both directions and Algorithm 2 is applied again until convergence or reaching Δ iterations. The final step of the procedure is to transform the obtained directed graph to undirected again and compute its MST.



Algorithm 1: Random k -neighbor Graph

Data: $Nodes, k$
Result: $Neighbors$: random k -neighbor
 $Neighbors \leftarrow \{\}$;
for x_i **in** $Nodes$ **do**
 $Neighbors(x_i) \leftarrow$ list of k random nodes from $Nodes$;
end

Table 2 Comparison of algorithms 3 and 4 using different data sets with $k = 20$, $\delta = 8$, $\Delta = \infty$ (optimization until convergence)

Data set (d, n)	Algorithm	# Iterations		Time (s)	Relative error %
		Phase 1	Phase 2		
MNIST	Alg. 3	14	–	27.8	1.67
	Alg. 4	8	3	14.8	1.63
Speech	Alg. 3	7	–	11.8	10.71
	Alg. 4	6	1	11.8	6.97
Celeba	Alg. 3	13	–	801	54.06
	Alg. 4	8	4	459	48.71
Shape	Alg. 3	25	–	236	16.66
	Alg. 4	8	7	80	10.11
Audio	Alg. 3	17	–	232	11.41
	Alg. 4	8	6	109	9.75
Unbalance	Alg. 3	27	–	36.3	3.52
	Alg. 4	8	3	4.20	5.86
Make blobs (2, 100,000)	Alg. 3	88	–	2022	22.11
	Alg. 4	8	11	180	16.92
Make blobs (1000, 50,000)	Alg. 3	12	–	333	0.373
	Alg. 4	8	2	237	0.372

The memory consumption of both methods can be adjusted by the number of edge distances that are stored during the computations. If the computed edge distances are stored in the memory, the time of the consecutive iterations reduces because fewer distances have to be computed in one iteration. On the other hand, the memory consumption continuously increases during the procedure. In the *long k-neighbor graph optimization* method at most $\Delta \cdot n \cdot k^2$ distances need to be stored and in the *short k-neighbor graph optimization* method, it depends on the structure of the data set. In our experiments, it was always less than the memory consumption of the *long k-neighbor graph optimization* method. It is observed that the second phase of the *short k-neighbor graph optimization* method, where the number of edges is $2n - 2$, converges rather fast in practice, thus its memory consumption and running time are much lower than the ones of the *long k-neighbor graph optimization* method.

If the edge distances are not stored during the computation, then the memory consumption is negligible ($O(n \cdot k)$) as only the indexes of k neighbors of each node are stored. However, the computation time increases with the repeated computation of the edge distances.

Essentially, the approach of the *long k-neighbor graph optimization* method resembles the approach used in [41], while the approach of the *short k-neighbor graph optimization* method was not considered before.

4 Numerical Examples

4.1 Applied Data Sets

To demonstrate the effectiveness of the proposed method, a series of experiments was conducted on various data sets. In total, 13 different data sets were utilized as shown in Table 1. Speech, MNIST, and Shuttle are from Multi-dimensional point data sets of the Outlier Detection Data Sets.¹ CelebA was introduced in Ref. [42] and can be accessed online.² Miss America, House, Europe, Unbalanced, and Birch can be found on the web.³ The first three of this group are real-world data, and the last two are synthetic data sets. Corel,

¹ <https://odds.cs.stonybrook.edu/#table1>.

² <https://paperswithcode.com/dataset/celeba>.

³ <https://cs.joensuu.fi/sipu/datasets/>.

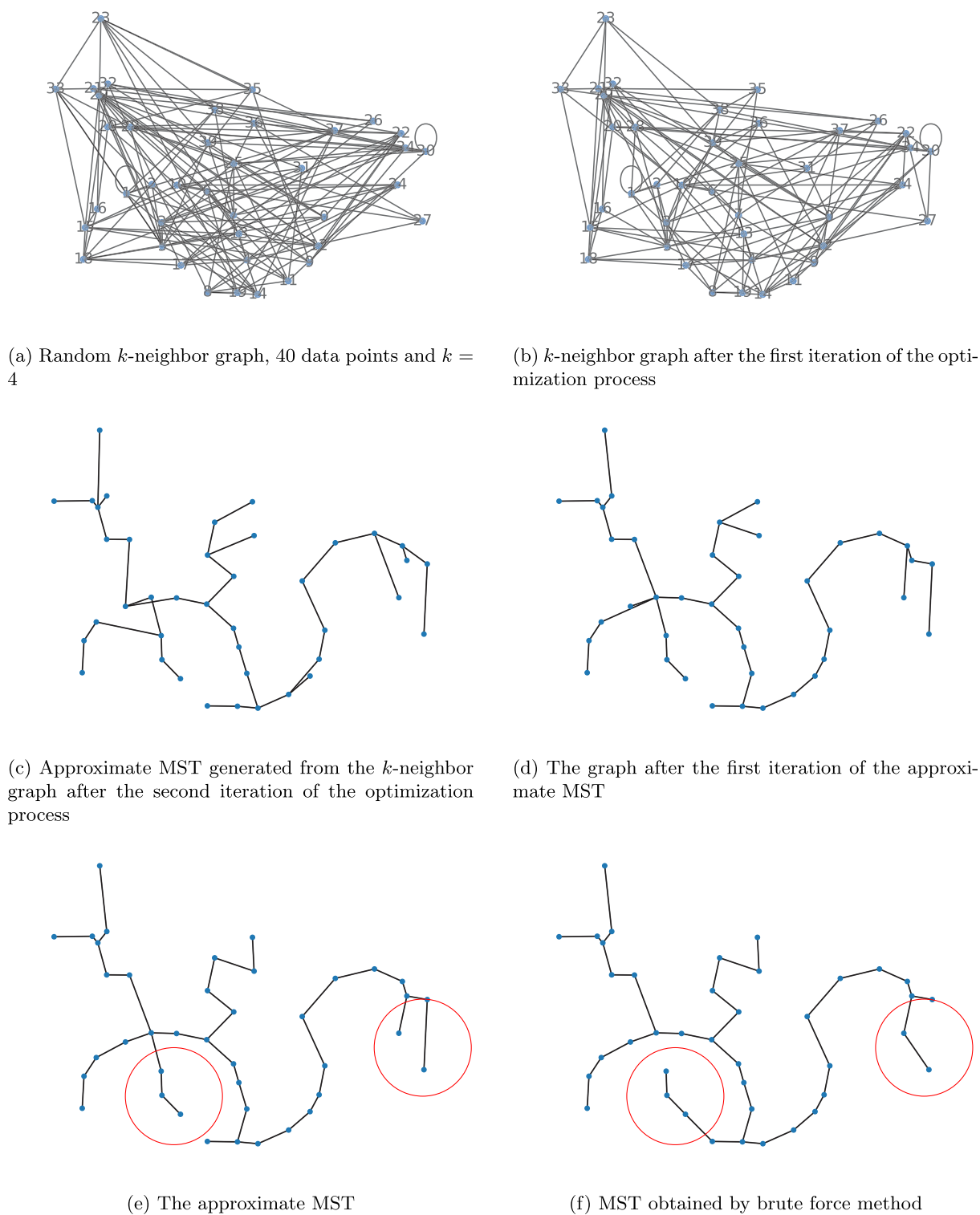


Fig. 1 Process of generating approximate MST based on Algorithm 4 and the associated MST. The circles indicate the differences between the obtained approximate MST compared to the MST

Algorithm 2: Optimizing the random graph**Data:** Random k neighbor list, Data set X **Result:** Optimized k neighbor list, $Edges$, $Converged$ $Edges \leftarrow []$; $Converged \leftarrow True$;**for** $x_i \in Nodes$ **do** **for** $x_j \in Neighbors(x_i)$ **do** $D \leftarrow distance(x_i, x_j)$; $\hat{D} \leftarrow D$; **for** $\hat{x}_z \in Neighbors(x_j)$ **do** **if** $\hat{x}_z \neq x_i$ **then** $D_{temp} \leftarrow distance(x_i, \hat{x}_z)$; **if** $D_{temp} < \hat{D}$ **and** $\hat{x}_z \notin Neighbors(x_i)$ **then** $\hat{D} \leftarrow D_{temp}$; $x_z \leftarrow \hat{x}_z$ **end** **end** **end** **if** $\hat{D} < D$ **then** $Neighbors(x_i) \leftarrow Neighbors(x_i) \setminus x_j$ $Neighbors(x_i) \leftarrow Neighbors(x_i) \cup x_z$ $Edges \leftarrow Edges \cup (x_i, x_z, \hat{D})$; $Converged \leftarrow False$ **else** $Edges \cup (x_i, x_j, D)$; **end**

 ▷ The Euclidean distance between a given node and its corresponding neighbor is denoted as D . Similarly, $\hat{D}$ represents the optimal Euclidean distance between the node and a neighbor of a neighbor. Here, x_z refers to the neighbor of a neighbor, while D_{temp} refers to the current Euclidean distance between the node and a neighbor of a neighbor.

end**end**

▷ Remove far neighbor;

▷ Add closer one;

Algorithm 3: Long k -neighbor graph optimization**Data:** X, k, Δ **Result:** MST $Random_k_neighbor \leftarrow Algorithm_1(Nodes, k)$; $Optimized_k_neighbor \leftarrow Random_k_neighbor$; $Epoch \leftarrow 1$; $Converged \leftarrow False$;**while** $Not\ Converged$ **and** $Epoch < \Delta$ **do** $Optimized_k_neighbor, Edges, Converged \leftarrow Algorithm_2(Optimized_k_neighbor, X)$; $Epoch \leftarrow Epoch + 1$;**end** $MST \leftarrow Exact_mst(n, Edges)$;

Shape, and Audio were used by [41, 43] and are available online.⁴ The *make_moons* library from *Sklearn data sets*⁵ was used to generate the Make_moons data set. Finally, the *make_blobs* library from *Sklearn data sets*⁶ was utilized to generate data sets of different sizes and dimensions.

⁴ <https://code.google.com/archive/p/nndes/downloads>.

⁵ https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html.

⁶ https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html.

4.2 Comparison of Short and Long k -Neighbor Graph Optimization Methods

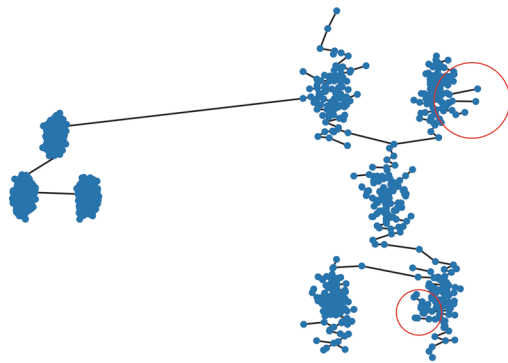
The run time and accuracy of the *short and long k -neighbor graph optimization* methods were compared using the data sets in Table 1, with hyperparameters $k = 20$, $\delta = 8$, and $\Delta = \infty$ in each case. Independently sampled initial random k -neighbor graphs were initiated for each comparison. To demonstrate the number of iteration cycles required for convergence, Δ was set to ∞ . This setup resulted in Algorithms 3 and phase 2 of Algorithms 4 running until the graph could



Algorithm 4: Short k -neighbor graph optimization

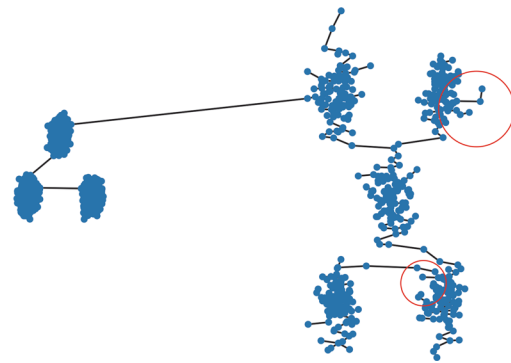
Data: X, k, Δ, δ
Result: MST
 $Random_k_neighbor \leftarrow Algorithm_1(Nodes, k);$
 $Optimized_k_neighbor \leftarrow Random_k_neighbor;$
 $Epoch \leftarrow 1;$
 $Converged \leftarrow False;$
while Not Converged and $Epoch < \delta$ **do**
 $Optimized_k_neighbor, Edges, Converged \leftarrow Algorithm_2(Optimized_k_neighbor, X);$
 $Epoch \leftarrow Epoch + 1;$
end
 $MST \leftarrow Exact_mst(n, Edges);$
while Not Converged and $Epoch < \Delta$ **do**
 $MST, Edges, Converged \leftarrow Algorithm_2(MST, X);$
 $MST \leftarrow Exact_mst(n, Edges);$
 $Epoch \leftarrow Epoch + 1;$
end

MST weight = 73.1467933624036



(a) Approximate MST obtained by Algorithm 4 with $k = 20$

MST weight = 71.75513157808092



(b) MST obtained by brute force

Fig. 2 Unbalanced 2d synthetic data set [44] with $n = 6500$ data points and 8 Gaussian clusters. Some differences between the approximate and the exact MSTs are marked with circles

no longer be improved, determined by the iteration cycle of Algorithm 2.

The obtained results, in Table 2, demonstrate that the *short k -neighbor graph optimization* method provides more accurate approximate MST and better execution time. Its lower execution time is due to the fact that it converges in fewer iterations because Algorithm 4 performs at most δ iterations of optimizing the random k -neighbor graph and then generates a directed graph with fewer edges whose optimization converges quickly. The intuitive explanation for the improvement in the total weight of the obtained approximate MST is that after optimizing the random k -neighbor graph, during the optimization of the approximate MST, the reverse edge of every existing edge in the approximate MST was added to create an undirected graph, which helps to avoid the convergence to local optima and enhances the accuracy of the final approximate MST.

The only data set for which optimization of the random k -neighbor graph converges within less than δ iterations is the Speech data set. In that case, the independent sampling of the initial random k -neighbor graph results in the differences in the number of required iterations for Algorithm 3 and 4.

4.3 Demonstration of the Behavior of Algorithm 4

Figure 1 presents an overview of the approximate MST generation process facilitated by Algorithm 4 for a data set generated by the *make_moons* algorithm from *Scikit-learn*, comprising only 40 data points, to enhance visual clarity. In the initial step, Algorithm 4 generates a random k -neighbor graph with $n = 40$ vertices and $n \cdot k = 160$ edges, assuming $k = 4$, which is depicted in Fig. 1a. (A higher value for k would result in a denser graph and potentially better



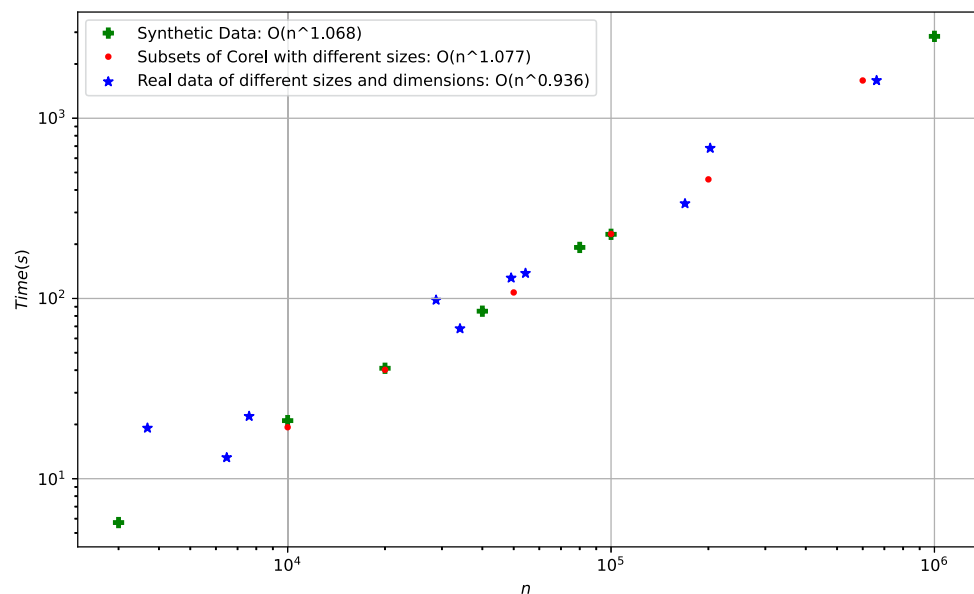
Table 3 Comparison against existing methods

Data Set (d, n)	MST_mlpack		FISHDBC		Alg. 4	
	Time (s)	Relative error %	Time (s)	Relative error %	Time (s)	Relative error %
MNIST	4.18	0	14.45	18.00	14.81	1.63
Speech	22.05	0	7.20	20.47	7.15	6.97
Celeba	10.43	0	671.68	96.12	459.00	48.71
Shape	115.31	0	85.13	38.08	80.00	10.11
Audio	360.78	0	267.42	14.10	109.00	9.75
Make blobs (64, 50,000)	270.70	0	249.62	8.57	141.50	2.94
Make blobs (128, 50,000)	615.00	0	244.40	5.84	142.01	1.82
Make blobs (256, 50,000)	1416.82	0	213.76	4.64	151.94	1.12
Make blobs (512, 50,000)	2870.95	0	230.18	2.76	161.79	0.74
Make blobs (1024, 50,000)	5405.67	0	216.17	2.42	181.67	0.48

Table 4 Execution time and memory consumption of obtaining ϵ -separated pairs using WSPD for different data sets from Table 1

Data set	n	d	$\epsilon = 2$		$\epsilon = 8$	
			Time (s)	Memory(GB)	Time (s)	Memory(GB)
Shuttle	49,097	9	13	6	96	35
MNIST	7603	100	21	7.5	28	9
Speech	3686	400	9	2	9	2
Celeba	202,599	39	Memory error			
Shape	28,775	544				
Audio	54,387	192				
Corel	662,317	14				
Corel (subset)	30,000	14				
Birch1	100,000	2	3	1.7	15.3	5
Europe	169,308	2	9	13	30	18

Memory error occurs after consuming all available memory (64 GB)

Fig. 3 Execution time (in seconds) using synthetic and real-world data sets where the dimension of data is shown in Table 1

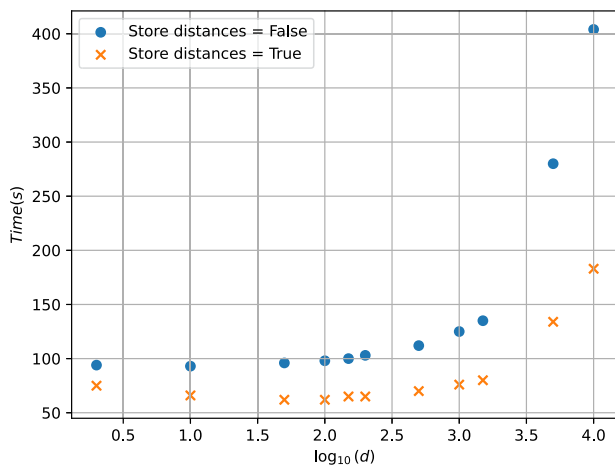
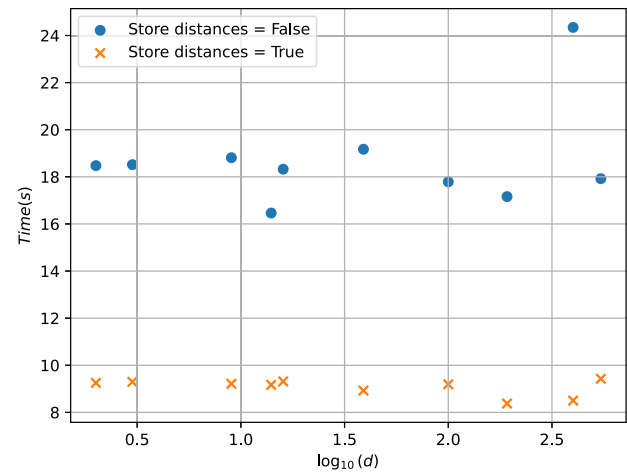

(a) Synthetic Make blobs data sets ($n = 20000$)

(b) Sub sets (first $n = 3500$ data points) of all real-world data sets from Table 1

Fig. 4 Impact of data dimension on the computational time with synthetic and real-world data sets ($k = 20$)

approximate MST, but with poorer graphical visibility). In this example, δ was set to 2 to execute only two iterations of random k -neighbor graph optimization due to the relatively small number of edges. The result of the first iteration is depicted in Fig. 1b. It is noticeable that the first iteration significantly reduced the number of long edges in the graph, leading to a visually less dense impression of the graph in Fig. 1b.

The primary approximate MST, as a result of the first phase of Algorithm 4, is obtained from the second iteration of the k -neighbor graph optimization, and it is depicted in Fig. 1c. Figure 1d depicts the result of the first optimization cycles of the second phase of Algorithm 4. This phase converges in 2 cycles; no further optimization is possible for this example. The MST generated from the optimized graph is provided in Fig. 1e. For comparison purposes, Fig. 1f presents the exact MST obtained using the brute force method. Most of the approximate and the exact MST are identical in Fig. 1e, f except for some minor differences indicated by circles.

As another example, Fig. 2 presents the results of Algorithm 4 using a 2D synthetic unbalanced data set generated by the Make blobs procedure containing 6500 data points, which are distributed among eight distinct clusters. It is worth noting that the size of this data set is relatively modest, and the primary objective is to demonstrate the efficacy of the proposed algorithm in handling unbalanced data sets. Similar to the previous example, Fig. 2 suggests that most of the approximate and the exact MST are identical with some minor exceptions, and the weights of the approximate and the exact MST are close.

4.4 Comparison of Algorithm 4 Against Existing Methods

The performance of Algorithm 4 was compared to two other algorithms, MST_mlpac [29], utilizing the code referenced in [45], and FISHDBC [35], utilizing the code repository specified by the author on GitHub.⁷ While MST_mlpac is an exact algorithm, FISHDBC and Algorithm 4 are approximate algorithms. Table 3 presents the results of numerical experiments using some of the benchmark data sets from Table 1. In most cases, Algorithm 4 provided accurate results in terms of relative error, where the relative error is calculated as follows:

$$\text{rel. error} = \frac{\text{approx. MST weight} - \text{MST weight}}{\text{MST weight}}$$

and the weight of an undirected graph is the sum of the weights of its edges. Algorithm 4 achieves superior performance for large data sets of high dimensions. As previously mentioned, algorithms that involve partitioning the space, such as using KD-tree, do not scale well with the dimensions of the data. This is evidenced in Table 3. The empirical running time of the MST_mlpac algorithm is significantly impacted by the number of dimensions in the data. While the MST_mlpac algorithm outperforms Algorithm 4 in two cases, namely when the number of dimensions is low (as in the Celeba dataset), or when the size of the dataset is small (as in the MNIST dataset), it is clear that this algorithm has limitations in terms of scalability. FISHDBC and Algorithm

⁷ <https://github.com/matteodellamico/flexible-clustering>.



4 demonstrate scalability with respect to the dimensionality of the data. However, Algorithm 4 exhibits superior performance in terms of both run time and relative error when compared to FISHDBC.

To the best of the authors' knowledge, there is currently no available implementation for obtaining an MST using sequential WSPD. We tried to run the implementation of [28] in a sequential manner. However, the current implementations shown in ⁸ only support low dimensions data (2–20). While an implementation for sequential WSPD exists,⁹ its output consists of a set of ϵ -separated pairs, as detailed in [33]. These pairs must be connected by an edge between the closest points for each pair using BCCP to obtain a connected graph from which an MST can be generated. The implementation of [33] was utilized to illustrate that WSPD has a strong dependence on the dimension of the data set because WSPD suffers for higher-dimensional data sets, as the number of pairwise comparisons that need to be stored increases exponentially, leading to higher memory requirements [33]. Therefore, the dimension of the data can significantly impact memory usage. Table 4 illustrates the execution time and memory usage of obtaining ϵ -separated pairs using the implementation of WSPD presented in [33]. It can be observed that WSPD is memory hungry when the dimension of the data set and ϵ are high. The execution time detailed in Table 4 indicates the required time to obtain ϵ -separated pairs. We note that the time required to connect the ϵ -separated pairs and to generate the approximate MST is not considered.

4.5 Empirical Time Complexity

To investigate the impact of the size of the data set on the running time of Algorithm 4, evaluations were conducted on both real-world and synthetic data sets of varying sizes. Figure 3 shows the results of the experiments conducted. A least squares fitting of the obtained data points suggests that the empirical computational time of Algorithm 4 is around $O(n^{1.07})$.

The performance of the proposed method on real-world data sets of different sizes and dimensions (all the real-world data sets in Table 1) follows the same trend as for the synthetic data sets. Additionally, subsets of the *Corel* data set were used to ensure the same structure but different sizes.

4.6 The Impact of Data Dimension

In the proposed approach, the dimension of the data set plays a role only in the computation of the edge weights of the

graph, i.e., the distance between data points. As a result, our approach is only slightly affected by the dimension of the data set compared to the algorithms that rely on spatial trees or partitioning the space into subsets.

To demonstrate this behavior, multiple experiments were performed using data sets of varying dimensions. The experiments were carried out in two distinct settings: One involved storing the calculated distances in the memory throughout the optimization process, while in the other, these distances were not stored.

Figure 4 plots the result of the experiments with and without storing the calculated distances using the synthetic and real-world data sets from Table 1. As expected, the execution time is higher when the calculated distances are not stored in the memory but are recalculated each time they are needed. The trend of the execution time as a function of the data set dimension suggests that the function that computes the distance has a significant overhead in our Python implementation, which dominates the running time for dimensions less than 10^3 . That is why the running time is seemingly independent of the dimension for dimensions less than 10^3 and starts increasing only after that limit.

5 Conclusion

Large data sets are often represented as graphs with as many nodes as the number of data points, and efficient graph procedures are needed to process such large graphs. One of the essential graph processing steps is the computation of the MST, which plays a role, e.g., in clustering, anomaly detection, etc.

Since exact methods are infeasible for large data sets, the paper presents an algorithm for approximate MST constructing accurately and computationally efficiently. The algorithm starts by creating a random k -neighbor graph and then optimizes it by crawling toward the optimal neighbors of each node to obtain an approximate k NN graph. Using this graph, the algorithm calculates an initial MST and optimizes it using the same crawling technique. Numerical experiments indicate the properties and the effectiveness of the proposed method.

Although the current paper primarily focuses on the static, sequential implementation of the proposed algorithm, it can be enhanced with dynamic parallel implementation, which makes the proposed algorithm compelling for a wide range of MST-based clustering tasks.

⁸ <https://github.com/wangyiqiu/hdbscan>.

⁹ https://github.com/dmatijev/wspd_pip.

Funding Open access funding provided by Budapest University of Technology and Economics.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Zahn, C.T.: Graph-theoretical methods for detecting and describing gestalt clusters. *IEEE Trans. Comput.* **100**(1), 68–86 (1971)
- Xu, D.; Tian, Y.: A comprehensive survey of clustering algorithms. *Ann. Data Sci.* **2**, 165–193 (2015)
- Jothi, R.; Mohanty, S.K.; Ojha, A.: Functional grouping of similar genes using eigenanalysis on minimum spanning tree based neighborhood graph. *Comput. Biol. Med.* **71**, 135–148 (2016)
- Mohapatra, C.; Ray, B.B.: A survey on large datasets minimum spanning trees. In: *International Symposium on Artificial Intelligence*, pp. 26–35. Springer, Berlin (2022)
- Juszczak, P.; Tax, D.M.; Pe, E.; et al.: Minimum spanning tree based one-class classifier. *Neurocomputing* **72**(7–9), 1859–1869 (2009)
- Zhong, C.; Miao, D.; Wang, R.: A graph-theoretical clustering method based on two rounds of minimum spanning trees. *Pattern Recogn.* **43**(3), 752–766 (2010)
- Zhong, C.; Miao, D.; Fränti, P.: Minimum spanning tree based split-and-merge: a hierarchical clustering method. *Inf. Sci.* **181**(16), 3397–3410 (2011)
- Wang, X.; Wang, X.L.; Chen, C.; et al.: Enhancing minimum spanning tree-based clustering by removing density-based outliers. *Digit. Signal Process.* **23**(5), 1523–1538 (2013)
- Grygorash, O.; Zhou, Y.; Jorgensen, Z.: Minimum spanning tree based clustering algorithms. In: *2006 18th IEEE International Conference on Tools with Artificial Intelligence (ICTAI'06)*, IEEE, pp. 73–81 (2006)
- Cheriton, D.; Tarjan, R.E.: Finding minimum spanning trees. *SIAM J. Comput.* **5**(4), 724–742 (1976)
- Stam, C.; Tewarie, P.; Van Dellen, E.; et al.: The trees and the forest: characterization of complex brain networks with minimum spanning trees. *Int. J. Psychophysiol.* **92**(3), 129–138 (2014)
- Sha'Abani, M.; Fuad, N.; Jamal, N.; et al.: KNN and SVM classification for EEG: a review. In: *InECCE2019: Proceedings of the 5th International Conference on Electrical, Control & Computer Engineering*, Kuantan, Pahang, Malaysia, 29th July 2019, Springer, Berlin, pp. 555–565 (2020)
- Taunk, K.; De, S.; Verma, S.; et al.: A brief review of nearest neighbor algorithm for learning and classification. In: *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, IEEE, pp. 1255–1260 (2019)
- Dhanabal, S.; Chandramathi, S.: A review of various k-nearest neighbor query processing techniques. *Int. J. Comput. Appl.* **31**(7), 14–22 (2011)
- Wang, M.; Xu, X.; Yue, Q.; et al.: A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search (2021). arXiv preprint [arXiv:2101.12631](https://arxiv.org/abs/2101.12631)
- Bentley, J.L.: Multidimensional binary search trees used for associative searching. *Commun. ACM* **18**(9), 509–517 (1975)
- Dolatshah, M.; Hadian, A.; Minaei-Bidgoli, B.: Ball*-tree: efficient spatial indexing for constrained nearest-neighbor search in metric spaces (2015). arXiv preprint [arXiv:1511.00628](https://arxiv.org/abs/1511.00628)
- Omohundro, S.M.: Five Balltree Construction Algorithms. *International Computer Science Institute, Berkeley* (1989)
- Jothi, R.; Mohanty, S.K.; Ojha, A.: Fast approximate minimum spanning tree based clustering algorithm. *Neurocomputing* **272**, 542–557 (2018)
- Wang, J.; Liu, W.; Kumar, S.; et al.: Learning to hash for indexing big data—A survey. *Proc. IEEE* **104**(1), 34–57 (2015)
- Jafari, O.; Maurya, P.; Nagarkar, P. et al.: A survey on locality sensitive hashing algorithms and their applications (2021). arXiv preprint [arXiv:2102.08942](https://arxiv.org/abs/2102.08942)
- Jin, Z.; Zhang, D.; Hu, Y.; et al.: Fast and accurate hashing via iterative nearest neighbors expansion. *IEEE Trans. Cybern.* **44**(11), 2167–2177 (2014)
- Shimomura, L.C.; Oyamada, R.S.; Vieira, M.R.; et al.: A survey on graph-based methods for similarity searches in metric spaces. *Inf. Syst.* **95**, 101507 (2021)
- Paredes, R.; Chávez, E.: Using the k-nearest neighbor graph for proximity searching in metric spaces. In: *String Processing and Information Retrieval: 12th International Conference, SPIRE 2005, Buenos Aires, Argentina, Nov. 2–4, 2005. Proceedings 12*, Springer, Berlin, pp. 127–138 (2005)
- McInnes, L.; Healy, J.; Astels, S.: HDBSCAN: hierarchical density based clustering. *J. Open Source Softw.* **2**(11), 205 (2017)
- Kershnerbaum, A.; Van Slyke, R.: Computing minimum spanning trees efficiently. *Proc. ACM Ann. Conf.* **1**, 518–527 (1972)
- Pettie, S.; Ramachandran, V.: An optimal minimum spanning tree algorithm. *JACM* **49**(1), 16–34 (2002)
- Wang, Y.; Yu, S.; Gu, Y.; et al.: Fast parallel algorithms for Euclidean minimum spanning tree and hierarchical spatial clustering. In: *Proceedings of the 2021 International Conference on Management of Data*, pp. 1982–1995 (2021)
- March, W.B.; Ram, P.; Gray, A.G.: Fast Euclidean minimum spanning tree: algorithm, analysis, and applications. In: *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 603–612 (2010)
- Narasimhan, G.; Zachariasen, M.: Geometric minimum spanning trees via well-separated pair decompositions. *J. Exp. Algorithm.* (JEA) **6**, 6 (2001)
- Callahan, P.B.; Kosaraju, S.R.: A decomposition of multidimensional point sets with applications to k-nearest-neighbors and n-body potential fields. *JACM* **42**(1), 67–90 (1995)
- Chan, T.M.: Well-separated pair decomposition in linear time? *Inf. Process. Lett.* **107**(5), 138–141 (2008)
- Matijević, D.: Well-separated pair decompositions for high-dimensional datasets. *Algorithms* **16**(5), 254 (2023)
- Prokopenko, A.; Sao, P.; Lebrun-Grandie, D.: A single-tree algorithm to compute the Euclidean minimum spanning tree on GPUS. In: *Proceedings of the 51st International Conference on Parallel Processing*, pp. 1–10 (2022)
- Dell'Amico, M.: Fishdbc: flexible, incremental, scalable, hierarchical density-based clustering for arbitrary data and distance (2019). arXiv preprint [arXiv:1910.07283](https://arxiv.org/abs/1910.07283)
- Malkov, Y.A.; Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **42**(4), 824–836 (2018)



37. Wang, X.; Wang, X.; Wilkes, D.M.: A divide-and-conquer approach for minimum spanning tree-based clustering. *IEEE Trans. Knowl. Data Eng.* **21**(7), 945–958 (2009)
38. Zhong, C.; Malinen, M.; Miao, D.; et al.: A fast minimum spanning tree algorithm based on k-means. *Inf. Sci.* **295**, 1–17 (2015)
39. Ma, Y.; Lin, H.; Wang, Y.; et al.: A multi-stage hierarchical clustering algorithm based on centroid of tree and cut edge constraint. *Inf. Sci.* **557**, 194–219 (2021)
40. Mishra, G.; Mohanty, S.K.: Efficient construction of an approximate similarity graph for minimum spanning tree based clustering. *Appl. Soft Comput.* **97**, 106676 (2020)
41. Dong, W.; Moses, C.; Li, K.: Efficient k-nearest neighbor graph construction for generic similarity measures. In: *Proceedings of the 20th International Conference on World Wide Web*, pp. 577–586 (2011)
42. Liu, Z.; Luo, P.; Wang, X. et al.: Deep learning face attributes in the wild. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 3730–3738 (2015)
43. Sieranoja, S.; Fränti, P.: Constructing a high-dimensional k NN-graph using a z-order curve. *J. Exp. Algorithm. (JEA)* **23**, 1–21 (2018)
44. Rezaei, M.; Fränti, P.: Set matching measures for external cluster validity. *IEEE Trans. Knowl. Data Eng.* **28**(8), 2173–2186 (2016)
45. Curtin, R.R.; Edel, M.; Shrit, O.; et al.: mlpack 4: a fast, header-only c++ machine learning library. *J. Open Source Softw.* **8**(82), 5026 (2023). <https://doi.org/10.21105/joss.05026>

